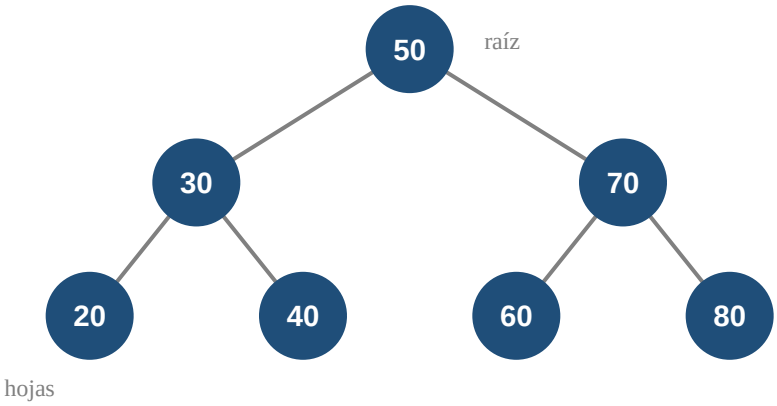
Árboles Binarios

Conceptos, tipos y recorridos

1. ¿Qué es un árbol binario?

Un **árbol binario** es una estructura de datos jerárquica formada por **nodos**, donde cada nodo tiene como máximo **dos hijos**: uno izquierdo y otro derecho. Se parte de un nodo especial llamado **raíz** y de él se ramifican los demás. A diferencia de las listas o arreglos, que son lineales, un árbol representa relaciones de jerarquía.



2. Terminología básica

- **Nodo:** elemento del árbol que guarda un dato y referencias a sus hijos.
- **Raíz:** nodo superior, el único sin padre.
- **Hijo izquierdo / derecho:** nodos conectados directamente debajo de otro nodo.
- **Hoja:** nodo sin hijos.
- **Subárbol:** un nodo junto con todos sus descendientes.
- **Nivel:** distancia desde la raíz (la raíz está en el nivel 0).
- **Altura:** número de niveles desde la raíz hasta la hoja más profunda.

3. Tipos de árboles binarios

Tipo	Descripción
Lleno	Cada nodo tiene 0 o 2 hijos.
Completo	Todos los niveles están llenos, salvo quizá el último, que se llena de izquierda a derecha.
Perfecto	Todos los nodos internos tienen 2 hijos y todas las hojas están al mismo nivel.
Degenerado	Cada nodo tiene un solo hijo; en la práctica se comporta como una lista.

De búsqueda (BST)	Para cada nodo, los valores del subárbol izquierdo son menores y los del derecho son mayores.

4. Recorridos

Recorrer un árbol significa visitar todos sus nodos en un orden determinado. Los recorridos en profundidad se definen según el momento en que se visita la raíz respecto a sus subárboles. Usando el árbol de ejemplo (50, 30, 70, 20, 40, 60, 80):

Recorrido	Orden	Resultado
InOrder	Izquierda, Raíz, Derecha	20 30 40 50 60 70 80
PreOrder	Raíz, Izquierda, Derecha	50 30 20 40 70 60 80
PostOrder	Izquierda, Derecha, Raíz	20 40 30 60 80 70 50
Por niveles	Nivel por nivel (BFS)	50 30 70 20 40 60 80

- **InOrder:** en un árbol de búsqueda entrega los valores ordenados de menor a mayor.
- **PreOrder:** útil para copiar un árbol o guardar su estructura, ya que la raíz aparece primero.
- **PostOrder:** útil para eliminar un árbol, pues los hijos se procesan antes que el padre.

5. Implementación en C++

```
struct Nodo {
    int dato;
    Nodo* izq;
    Nodo* der;
    Nodo(int v) : dato(v), izq(nullptr), der(nullptr) {}
};

void inOrder(Nodo* r) {
    if (!r) return;
    inOrder(r->izq);
    cout << r->dato << " ";
    inOrder(r->der);
}

void preOrder(Nodo* r) {
    if (!r) return;
    cout << r->dato << " ";
    preOrder(r->izq);
    preOrder(r->der);
}
```

El código completo está en el archivo **arbol_binario.cpp**.

6. Complejidad (árbol de búsqueda)

Operación	Promedio	Peor caso

Búsqueda	O(log n)	O(n)
Inserción	O(log n)	O(n)
Eliminación	O(log n)	O(n)
Recorrido completo	O(n)	O(n)

El peor caso ocurre cuando el árbol está desbalanceado (por ejemplo, al insertar valores ya ordenados se obtiene un árbol degenerado). Los árboles autobalanceados, como AVL o Rojo-Negro, garantizan O(log n).

7. Aplicaciones

- Búsqueda y ordenamiento eficiente de datos (árboles de búsqueda).
- Evaluación de expresiones matemáticas (árboles de expresión).
- Compresión de datos (árboles de Huffman).
- Colas de prioridad (heaps, que son árboles binarios completos).
- Índices en bases de datos y sistemas de archivos.